CTD Python Essentials

Week 6: Data Cleaning and Validation

Pivot Tables

- Reorganize and summarize data to analyze relationships and discover insights
- Aggregate data across categories (sum, count, etc.)
- Group rows and columns based a one or more keys
- Reshape data to make comparison more intuitive
- Quickly compare categories across different dimensions

```python
data = {
    'Store': ['North', 'North', 'North', 'South', 'South', 'South'],
    'Product': ['Apple', 'Banana', 'Orange', 'Apple', 'Banana', 'Orange'],
    'Sales': [120, 75, 50, 130, 70, 80],
    'Quantity': [30, 25, 10, 35, 20, 15]
}
```

```python
# See total sales of each product by store.
# pivot on Store (as rows), Product (as columns), and use the sum of Sales
pivot_sales = pd.pivot_table(df,
                             index='Store',
                             columns='Product',
                             values='Sales',
                             aggfunc='sum')
```

Product	Apple	Banana	Orange
Store			
North	120	75	50
South	130	70	80

More Pivot Table Examples

```python
# multiple statistics
pivot_multi = pd.pivot_table(df,
                             index='Store',
                             columns='Product',
                             values='Sales',
                             aggfunc=['sum', 'mean'])
```

```python
# Multiple indices
# Pivot on Quantity with Store and Product as row indices
pivot_multi_index = pd.pivot_table(df,
                                   index=['Store', 'Product'],
                                   values='Quantity',
                                   aggfunc='sum')
```

	sum			mean		
Product	Apple	Banana	Orange	Apple	Banana	Orange
Store						
North	120	75	50	120.0	75.0	50.0
South	130	70	80	130.0	70.0	80.0

		Quantity
Store	Product	
North	Apple	30
	Banana	25
	Orange	10
South	Apple	35
	Banana	20
	Orange	15

Using apply()

```
df = pd.DataFrame({
    'A': [10, 20, 30],
    'B': [4, 5, 6]
})
```

	A	B
0	10	4
1	20	5
2	30	6

- Apply a function along an axis of a pandas DataFrame or Series
  - axis=0 for columns, axis=1 for rows
- Row-wise or column-wise transformations, aggregations, or custom logic that goes beyond built-in methods (like sum, mean, etc.)

```
# Returning multiple columns
def compute_ratios(row):
    total = row['A'] + row['B']
    # Return a Series
    return pd.Series({
        'A_ratio': row['A'] / total,
        'B_ratio': row['B'] / total
    })
df[['A_ratio', 'B_ratio']] = df.apply(compute_ratios, axis=1)
```

	A	B	A_ratio	B_ratio
0	10	4	0.714286	0.285714
1	20	5	0.800000	0.200000
2	30	6	0.833333	0.166667

Handling Missing Data

- isnull() AKA isna()
  - Boolean same-sized object, DataFrame or Series (True for NA/Null)
    - Use .any(axis=1) for rows
- dropna()
  - Return a DataFrame with NA entries dropped from it or None if inplace=True, default axis=0 (index or rows)
- fillna()
  - Fill NA values with the specified value
- See also ffill(), bfill(), interpolate()

Data Transformation

- Specific datatype using astype()
- Numeric type using to_numeric()
- Dates using to_datetime()
- Using map() – apply any function
- Using replace() – replace specific values
  - Can use regex
  - Can use a dict() to specify alternative replacement values

```
df = pd.DataFrame({
    "A": ["10", "20", "30", "40"],
    "B": ["1.5", "2.5", "3.6", "4.1"],
    "C": ["2021-01-01", "2021-02-15", "2021-03-10", "2021-04-20"],
    "D": ["apple", "banana", "cherry", "banana"],
    "E": ['Beantown', 'Cisco', 'Windy', 'Philly']
})
```

	A	B	C	D	E
0	10	1.5	2021-01-01	apple	Beantown
1	20	2.5	2021-02-15	banana	Cisco
2	30	3.6	2021-03-10	cherry	Windy
3	40	4.1	2021-04-20	banana	Philly

```
df["A"] = df["A"].astype(int)
df["B"] = pd.to_numeric(df["B"])
df["C"] = pd.to_datetime(df["C"])
df["D"] = df["D"].map(lambda fruit: fruit.upper())
df["E"] = df["E"].replace({'Beantown': 'Boston', 'Cisco': 'San Francisco',
'Windy': 'Chicago', 'Philly': 'Philadelphia'})
```

	A	B	C	D	E
0	10	1.5	2021-01-01	APPLE	Boston
1	20	2.5	2021-02-15	BANANA	San Francisco
2	30	3.6	2021-03-10	CHERRY	Chicago
3	40	4.1	2021-04-20	BANANA	Philadelphia

Data Discretization

- Use cut() to discretize into user specified bins
  - Can specify a list of bins or number of bins
  - Can specify labels for the bins
- Use qcut() to discretize into user specified quatiles

```
data = {
    'age': [15, 20, 22, 23, 29, 34, 37, 45, 52, 63, 75, 87]
}
# cut with specified bins
df['age_bin'] = pd.cut(df['age'], bins=[0, 18, 35, 60, 100],
                       labels=['Child', 'Young Adult', 'Adult', 'Senior'])
# qcut using specified quantiles
dfn['age_3bins'] = pd.qcut(dfn['age'], [0, .15, .4, .75, 1.],
                           labels=['Child', 'Young Adult', 'Adult', 'Senior'])
```

	age	age_bin
0	15	Child
1	20	Young Adult
2	22	Young Adult
3	23	Young Adult
4	29	Young Adult
5	34	Young Adult
6	37	Adult
7	45	Adult
8	52	Adult
9	63	Senior
10	75	Senior
11	87	Senior

	age	age_qbins
0	15	Child
1	20	Child
2	22	Young Adult
3	23	Young Adult
4	29	Young Adult
5	34	Adult
6	37	Adult
7	45	Adult
8	52	Adult
9	63	Senior
10	75	Senior
11	87	Senior

Deduplication

- The drop_duplicates() method can be used to remove duplicate rows from a dataFrame
  - Can limit to specific columns using the subset parameter
  - Set ignore_index=True to reset the index

```
data = {
    'Name': ['Alice', 'Bob', 'Alice', 'Charlie', 'Bob'],
    'Age': [24, 42, 24, 36, 42],
    'City': ['New York', 'Los Angeles', 'New York', 'Chicago', 'Pittsburgh']
}
df = pd.DataFrame(data)
print(f'Original\n {df}')
new_df = df.drop_duplicates(subset=['Name', 'Age'])
print(f'new\n {new_df}')
```

```
      Name  Age         City
0    Alice   24     New York
1      Bob   42  Los Angeles
2    Alice   24     New York
3  Charlie   36      Chicago
4      Bob   42   Pittsburgh
```

```
      Name  Age         City
0    Alice   24     New York
1      Bob   42  Los Angeles
3  Charlie   36      Chicago
```

Handling Outliers

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Hannah'],
    'Age': [25, 30, 35, 40, 22, 29, 34, 128],
    'Weight': [55, 300, 68, 80, 50, 72, 65, 60],
    'Height': [165, 175, 170, 217, 160, 168, 172, 165]
}
```

```
df['Age'] = df['Age'].apply(lambda x: df['Age'].median() if x > 110 else x)
df['Weight'] = df['Weight'].apply(lambda x: df['Weight'].mean() if x > 120 else x)
df['Height'] = df['Height'].apply(lambda x: df['Height'].median() if x > 210 else x)
```

- The apply() method can be used to replace outliers with a reasonable value (e.g. mean, median, mode).
- For columns which follow a Gaussian Distribution outliers are often defined in terms of the mean and standard deviation.

```
age_series = df['Age']
mean = age_series.mean()
std_dev = age_series.std()
cutoff = 2 * std_dev
lower_bound = mean - cutoff
upper_bound = mean + cutoff
print(f'lower bound: {lower_bound:.2f} '
      f'upper bound: {upper_bound:.2f}')
```

	Name	Age	Weight	Height
0	Alice	25	55	165
1	Bob	30	300	175
2	Charlie	35	68	170
3	David	40	80	217
4	Eve	22	50	160
5	Frank	29	72	168
6	Grace	34	65	172
7	Hannah	128	60	165

	Name	Age	Weight	Height
0	Alice	25.0	55.00	165.0
1	Bob	30.0	93.75	175.0
2	Charlie	35.0	68.00	170.0
3	David	40.0	80.00	169.0
4	Eve	22.0	50.00	160.0
5	Frank	29.0	72.00	168.0
6	Grace	34.0	65.00	172.0
7	Hannah	32.0	60.00	165.0

Q&A and Demo

Example code:
Python 100 Week 6 Examples